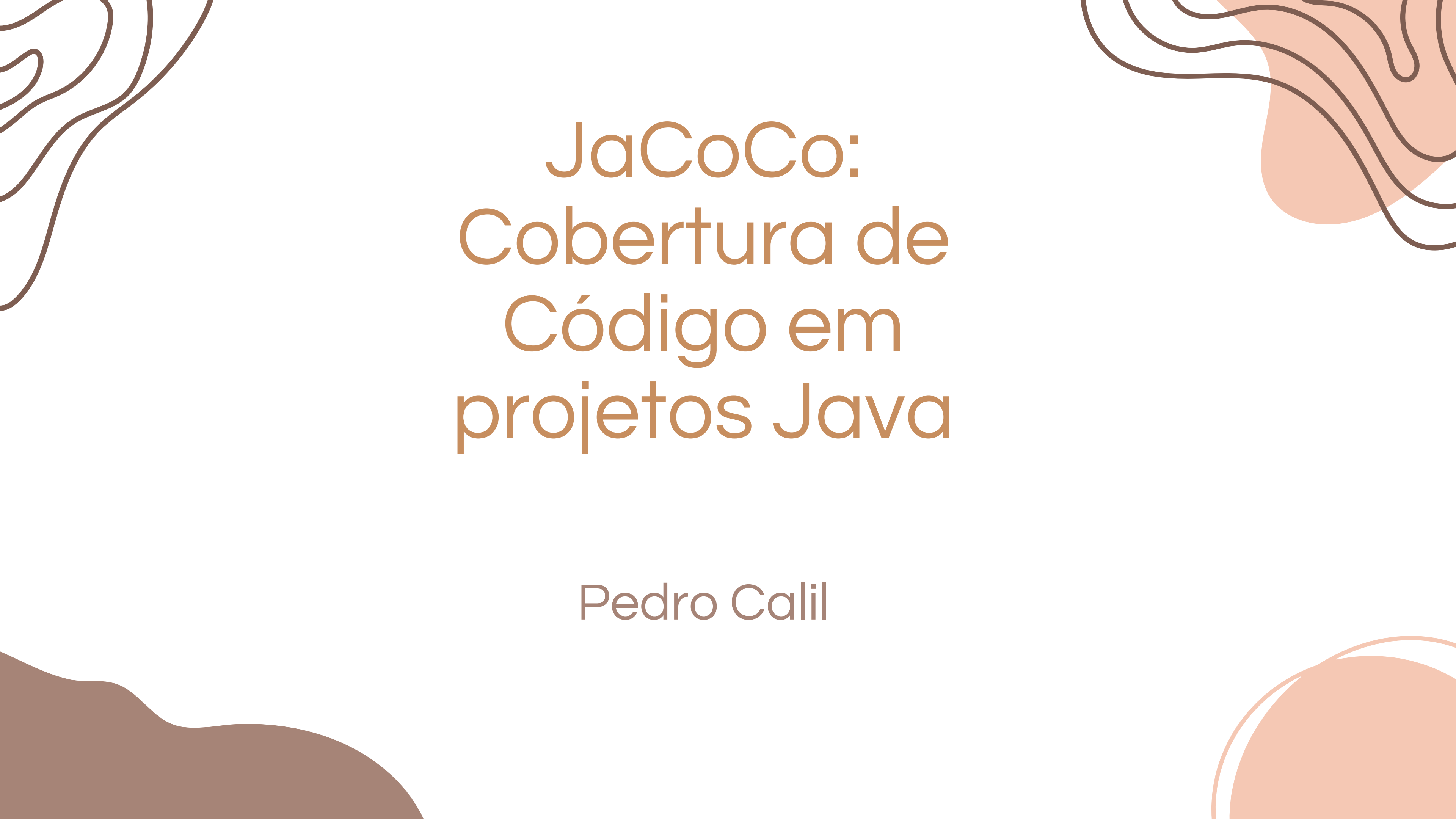




JaCoCo: Cobertura de Código em projetos Java

Pedro Calil

O que é Code Coverage?

1

Métrica que indica quanto do código foi executado pelos testes.

2

Ajuda a encontrar caminhos não testados e orientar prioridades de teste.

3

Cobertura $\neq$ qualidade: mostra o que foi executado, não se o teste pegaria um bug (assertividade)

Contadores do JaCoCo (o que ele mede)

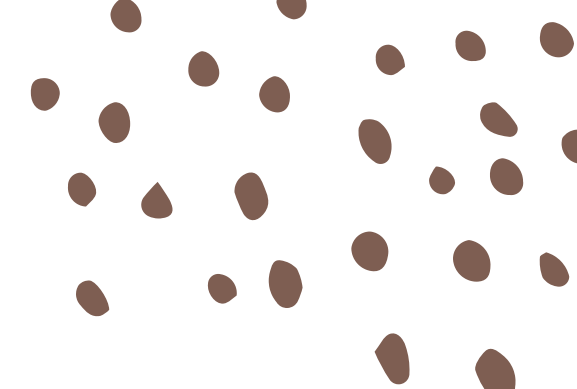
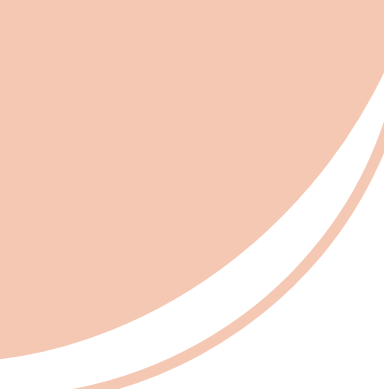
INSTRUCTION (C0)
— fração de
bytecodes
executados.

BRANCH (C1) —
fração de
ramos/decisões
exercitadas.

LINE — linhas
executadas/total.

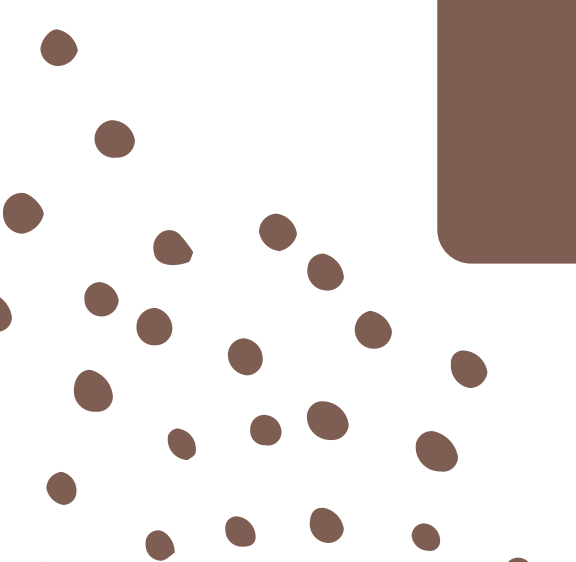
METHOD & CLASS —
métodos/classes tocados.

COMPLEXITY — aproxima
ciclomatica (nº de decisões).

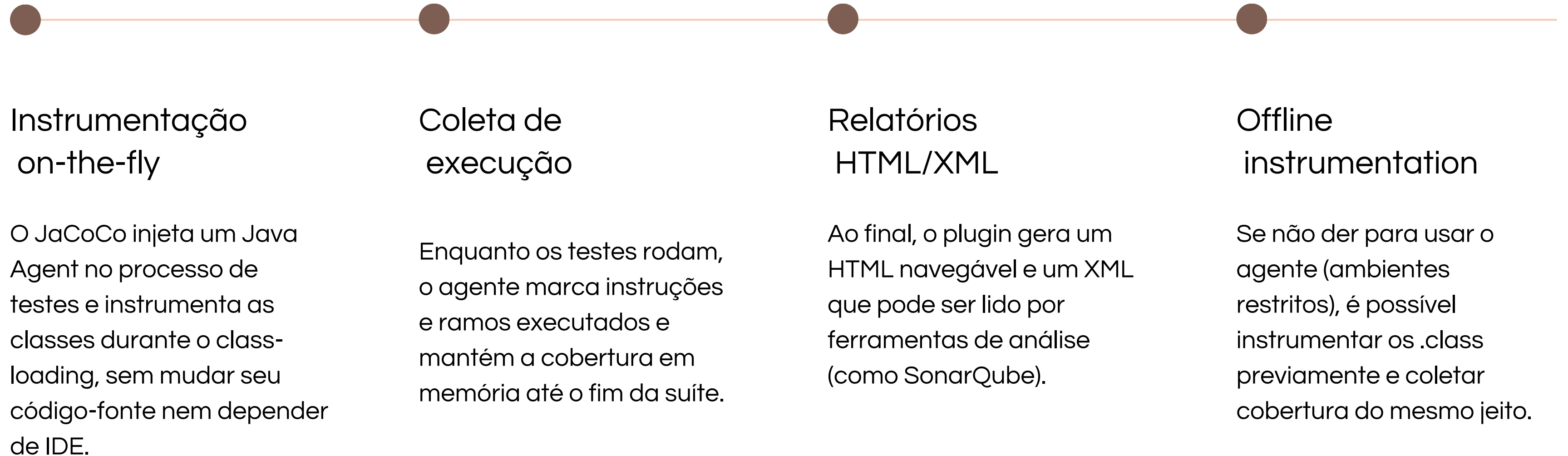


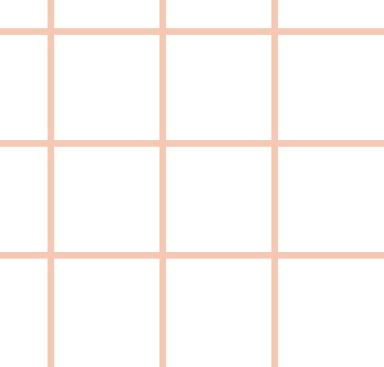
Onde fica na pirâmide de testes?

É uma métrica típica de caixa-branca que atua principalmente em testes de unidade (base da pirâmide), mas também funciona em testes de integração e até E2E em JVM via Java Agent.



Como o Jacoco funciona?





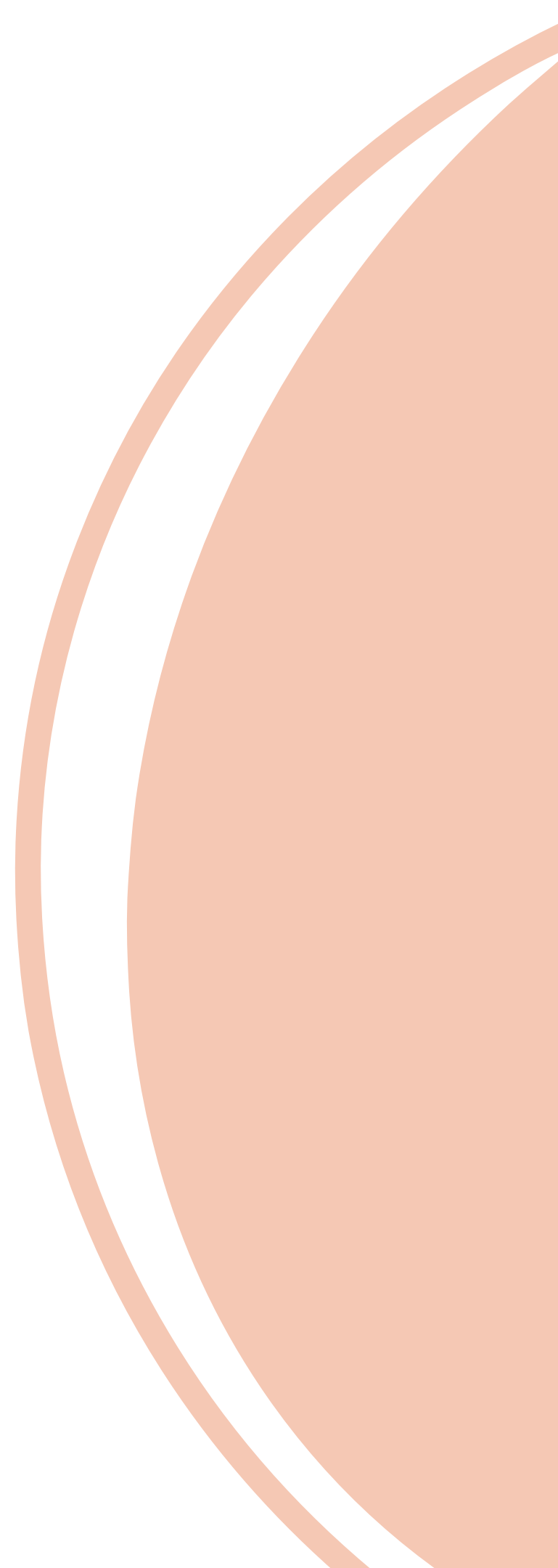
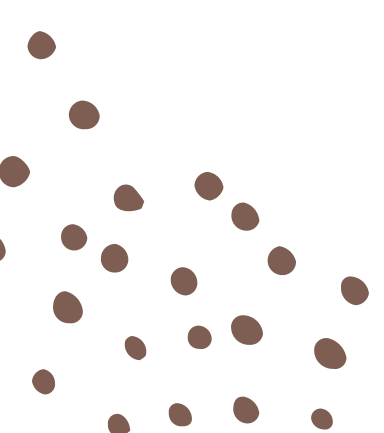
Setup com Maven (plugin jacoco).

1

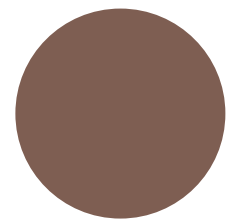
“Prepare-agent” injeta o agente na fase de testes.

2

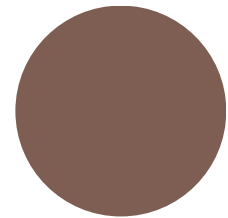
“Report” cria HTML/XML na fase verify.



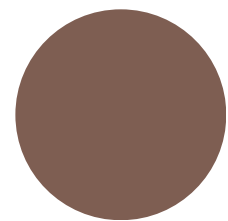
Projeto de Demonstração



Classe TriangleClassifier (regras de triângulo).

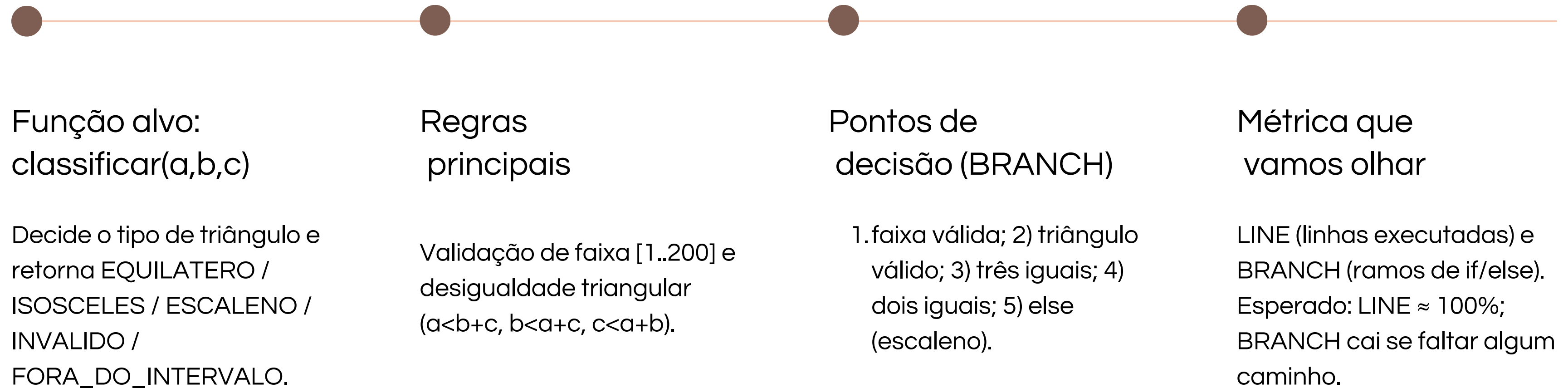


Testes JUnit cobrindo: Equilátero, Isósceles, Escaleno, Inválido, Fora do Intervalo.



Exercita caminhos e ramos para ilustrar BRANCH x LINE.

Alvo da cobertura: Classificar (a,b,c)





```
1  @Test
2      void foraDoIntervalo() {
3          assertEquals(Tipo.FORA_DO_INTERVALO, TriangleClassifier.classificar(0, 10, 10));
4          assertEquals(Tipo.FORA_DO_INTERVALO, TriangleClassifier.classificar(10, 201, 10));
5      }
6
7  @Test
8  void invalidoDesigualdadeTriangular() {
9      assertEquals(Tipo.INVALIDO, TriangleClassifier.classificar(1, 2, 3)); // 1 + 2 == 3
10     assertEquals(Tipo.INVALIDO, TriangleClassifier.classificar(1, 10, 20));
11 }
12
13 @Test
14 void equilatero() {
15     assertEquals(Tipo.EQUILATERO, TriangleClassifier.classificar(5, 5, 5));
16 }
17
18 @Test
19 void isosceles() {
20     assertEquals(Tipo.ISOSCELES, TriangleClassifier.classificar(5, 5, 6));
21     assertEquals(Tipo.ISOSCELES, TriangleClassifier.classificar(6, 5, 5));
22     assertEquals(Tipo.ISOSCELES, TriangleClassifier.classificar(5, 6, 5));
23 }
24
25 @Test
26 void escaleno() {
27     assertEquals(Tipo.ESCALENO, TriangleClassifier.classificar(4, 5, 6));
28 }
```

. Faixa inválida → 1º if
(ramo true).

. Desigualdade falha → 2º
if (ramo true).

. Equilátero → 3º if (true).

. Isósceles → 4º if (true)

. Escaleno → else final
(false do 4º if).

Execução

```
~/.../pedro-calil/src  
○ > cd workshops/submissions/pedro-calil/src && mvn -DskipTests=false clean verify
```

1

Prepare-agent injeta o Java Agent nos testes.

2

Report cria o HTML/XML em verify.

3

Check aplica o quality gate (quebra se < mínimo).

Como ler o relatório HTML do JaCoCo

index.html → stsw.jacoco → TriangleClassifier → Source

. **Instruction**: % de bytecodes.

. **Cxty**: complexidade (decisões).

. **Branches**: % de ramos (if/switch).

. **Verde** = executada · **Vermelho** = não.

TriangleClassifier.java

```
1. package stsw.jacoco;
2.
3. public class TriangleClassifier {
4.
5.     public enum Tipo {
6.         EQUILATERO, ISOSCELES, ESCALENO, INVALIDO, FORA_DO_INTERVALO
7.     }
8.
9.     /**
10.      * Classifica um triângulo a partir dos lados a, b, c.
11.      * Regras do curso:
12.      * - Cada lado deve estar no intervalo [1, 200].
13.      * - Desigualdade triangular: a < b + c, b < a + c, c < a + b.
14.      */
15.     public static Tipo classificar(int a, int b, int c) {
16.         if (!entrele200(a) || !entrele200(b) || !entrele200(c)) {
17.             return Tipo.FORA_DO_INTERVALO;
18.         }
19.         if (!trianguloValido(a, b, c)) {
20.             return Tipo.INVALIDO;
21.         }
22.         if (a == b && b == c) return Tipo.EQUILATERO;
23.         if (a == b || a == c || b == c) return Tipo.ISOSCELES;
24.         return Tipo.ESCALENO;
25.     }
26.
27.     static boolean trianguloValido(int a, int b, int c) {
28.         // Use < (e não <=) para evitar soma igual ao terceiro lado
29.         return a < b + c && b < a + c && c < a + b;
30.     }
31.
32.     static boolean entrele200(int x) {
33.         return x >= 1 && x <= 200;
34.     }
35. }
```

← xdg-open target/site/jacoco/stsw.jacoco/TriangleClassifier.java.html

xdg-open target/site/jacoco/index.html

STSW JaCoCo Demo

Element ↕	Missed Instructions ↕	Cov. ↕	Missed Branches ↕	Cov. ↕	Missed ↕	Cxty ↕	Missed ↕	Lines ↕	Missed ↕	Methods ↕	Missed ↕	Classes ↕
 stsw.jacoco		97%		89%	4	19	1	12	1	5	0	2
Total	3 of 104	97%	3 of 28	89%	4	19	1	12	1	5	0	2

Quality Gate: jacoco:check

ANTES: Limites definidos no pom.xml

- Limites no pom.xml: BRANCH ≥ 0.80 · LINE ≥ 0.90
- Todos os testes ativos
- Resultado: build passa

```
> mvn -DskipTests=false clean verify
[INFO] Loading execution data file /home/pca
kshops/submissions/pedro-calil/src/target/ja
[INFO] Analyzed bundle 'STSW JaCoCo Demo' wi
[INFO]
[INFO] --- jacoco-maven-plugin:0.8.13:check
[INFO] Loading execution data file /home/pca
kshops/submissions/pedro-calil/src/target/ja
[INFO] Analyzed bundle 'jacoco-demo' with 2
[INFO] All coverage checks have been met.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```


Quality Gate: jacoco:check

DEPOIS (falha):

- Comentei 1 teste que cobre decisão
- Rodei o mesmo comando
- Resultado: build falha (jacoco:check)

```
1  /* @Test
2      void isosceles() {
3          assertEquals(Tipo.ISOSCELES, TriangleClassifier.classificar(5, 5, 6));
4          assertEquals(Tipo.ISOSCELES, TriangleClassifier.classificar(6, 5, 5));
5          assertEquals(Tipo.ISOSCELES, TriangleClassifier.classificar(5, 6, 5));
6      } */
```

```
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 2.328 s
[INFO] Finished at: 2025-10-03T03:41:58-03:00
[INFO] -----
[ERROR] Failed to execute goal org.jacoco:jacoco-maven-plugin:0.8.13:check (check) on project jacoco-demo: Coverage checks
have not been met. See log for details. -> [Help 1]
```

Ferramentas similares (a nível de cobertura de código).

OpenClover (Java)

- Relatórios ricos, cobertura por ramo/declaração, recursos de “test optimization”.
- Setup mais trabalhoso; comunidade menor hoje.

Kover (Kotlin)

- Cobertura para projetos Kotlin (muitas vezes com engine IntelliJ/JaCoCo), bom para multiplatform.
- Ecossistema ainda evoluindo; relatórios/integrações variam.

Cobertura (Java, legado)

- Simples, tradicional.
- Compatibilidade limitada com Java moderno; manutenção baixa; instrumentação offline.

Todos atuam no nível de cobertura (white-box).

Vantagens

- Padrão no Java: todo ecossistema conhece/suporta.
- Configuração simples: Maven/Gradle + JUnit → mvn verify e pronto.
- Relatório claro: HTML mostra linhas e ramos faltando (verde/vermelho/losango).
- Quality Gate: dá pra falhar o build se a cobertura cair.

Desvantagens

- Cobertura ≠ qualidade: dá para “bater meta” sem testar bem.
- Ajustes de exclusão: código gerado/boilerplate pode distorcer a %.
- BRANCH confunde no início: linha pode estar alta e ramo baixo (faltou else/caso alternativo).
- Não mede assertividade: só conta o que rodou, não se o teste pegaria um bug.
-

Cases de sucesso

- . Apache Commons Lang (ASF): Publica no site oficial o “JaCoCo Coverage Report” como parte dos relatórios do projeto.
- . Quarkus (Red Hat): Tem extensão oficial “quarkus-jacoco” e guia de uso; cobertura é prática recomendada na documentação.
- . Palantir: Mantém o plugin gradle-jacoco-coverage para gates de cobertura e relatório agregado em projetos multi-módulo.

Conclusão

- Quando usar: projetos Java com testes automatizados e CI.
- Quando evitar: se o time confunde métrica com qualidade; foco só em E2E.